

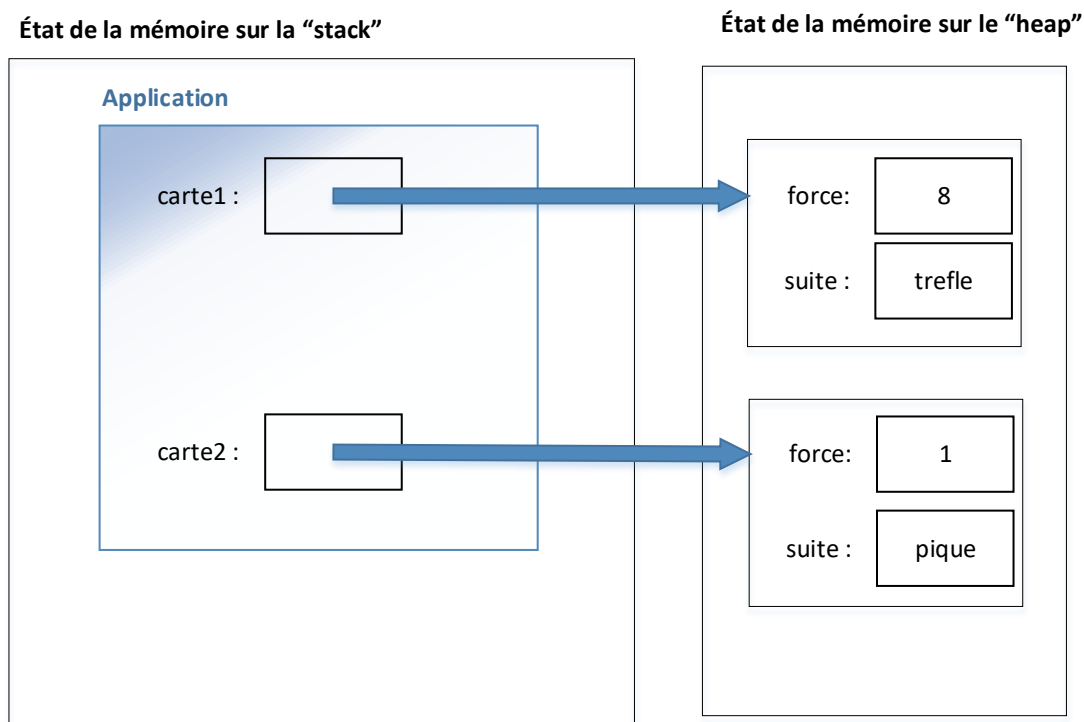
Cours 02 – Notion 5 : Passage par copie vs Passage par référence

2.5.1 Définitions

Lorsqu'une instance d'une classe est créée, cette instance n'est pas placée dans la "stack" en mémoire. Elle est plutôt placée dans le "heap". Cependant, les deux variables associées aux instances sont bel et bien dans le "stack".

```
// Création de deux instances.
//
carte1 = Carte(8, Carte.Couleur.trefle);    %Le 8 de trèfle.
carte2 = Carte(1, Carte.Couleur.pique);    %L'as de pique.
```

Ainsi, les propriétés du 8 de trèfle et les propriétés de l'as de pique sont dans le heap, alors que carte1 et carte2 sont dans le "stack".



Le mécanisme de référence aura un impact sur :

- le rôle des paramètres formels ;
- la syntaxe des appels de méthodes ;
- L'assignation et la copie des instances.

2.5.2 La référence this

Lors de l'appel d'une méthode, l'objet sur laquelle la méthode est appelée est référencé par l'objet *this*. L'action de cette méthode modifie donc les attributs de l'objet sur laquelle cette méthode est appelée. Et on accède aux attributs de cet objet grâce à la référence *this* telle que définie dans les multiples exemples de la capsule 2.4.

```
// Création de deux instances.  
//  
carte1 = Carte(8, Carte.Couleur.trefle);    %Le 8 de trèfle.  
carte2 = Carte(1, Carte.Couleur.pique);    %L'as de pique.  
  
// Modification des deux instances.  
carte1.setForce(1);  
carte2.setSuite(Carte.Couleur.carreau);
```

Il est important de comprendre que, même s'il n'y a pas de valeur de retour associée à cet appel, **l'instance sur laquelle une méthode est appelée est automatiquement transmis en référence** par l'entremise du terme *this*.

- Ainsi, la référence "this" dans "setSuite" reçoit l'instance "carte1".

Alors, que le paramètre formel "nouvelleForce" de "setSuite" reçoit la valeur 1.

```
Carte1.setForce(1):  
  
public void setForce(int nouvelleForce)  
{  
    this.force = nouvelleForce;  
}
```

Les autres paramètres d'une méthode continuent d'être copiés selon leur ordre de présentation.

2.5.3 Passage par valeur

Le passage par valeur s'applique aux paramètres de type primitifs. Dans ces situations, lorsqu'une donnée est utilisée comme un paramètre effectif, cette donnée est copiée dans la fonction et la copie est assigné au paramètre formel. La valeur du paramètre ne peut être modifié dans une fonction puisque la fonction reçoit une copie de la valeur initiale.

(exemple prochaine page)

```

Carte creerCarte(int noCarte)
{
    int force;
    int suite;

    //Tant que nous ne sommes pas en présence d'une force entre 1 et 13,
    //On recule le noCarte d'une couleur pendant que l'on ajuste cette couleur.
    suite = 1;
    while(noCarte > 13)
    {
        noCarte = noCarte - 13;
        suite++;
    }

    //On conserve la force trouvée.
    force = noCarte;

    return new Carte(force, suite);
}

public static void main()
{
    int force = 0;
    int suite = 0;

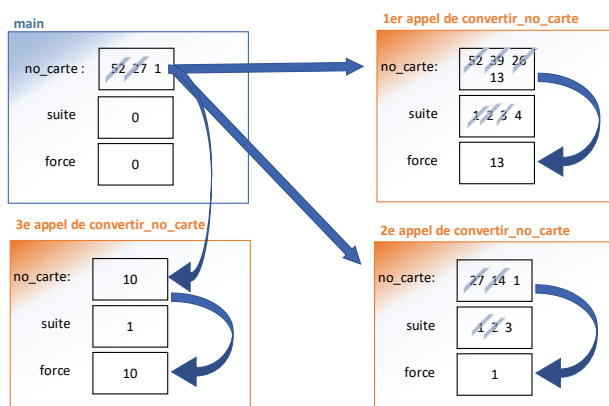
    int noCarte = 52;
    Carte c1 = creerCarte(noCarte);

    noCarte = 27;
    Carte c2 = creerCarte(noCarte);

    noCarte = 10;
    Carte c3 = creerCarte(noCarte);
}

```

En observant l'illustration suivante, on remarque que les trois valeurs de "noCarte", employé en tant que paramètres effectifs, sont successivement **copiées** dans un espace mémoire associé à chaque appel de fonction. Les modifications faites à l'intérieur de chaque appel n'ont aucune incidence quant aux valeurs des variables contenues dans le "main", que ce soit pour "noCarte", "suite" ou "force".



On en déduit que le passage par copie crée une deuxième instance d'une variable et qu'une modification à cette variable n'est visible que dans la fonction où la modification a lieu. Le "main" n'a jamais conscience de cette modification.

2.5.4 Passage par référence

Tel que mentionné en 2.5.1, lorsqu'un objet est créé, il est placé dans le "heap" et la "stack" du "main" contient une référence vers cet objet.

```
public static void main()
{
    Etudiant toi = new Etudiant('Marc-Antoine', 97.5);
    System.out.println(toi);

    toi.setNote(100);
    System.out.println(toi);
}
```

Application ×

C:\Users\Francis\.jdk\openjdk-18.0.1.1\bin\java.exe
Marc-Antoine - 97.5%
Marc-Antoine - 100.0%

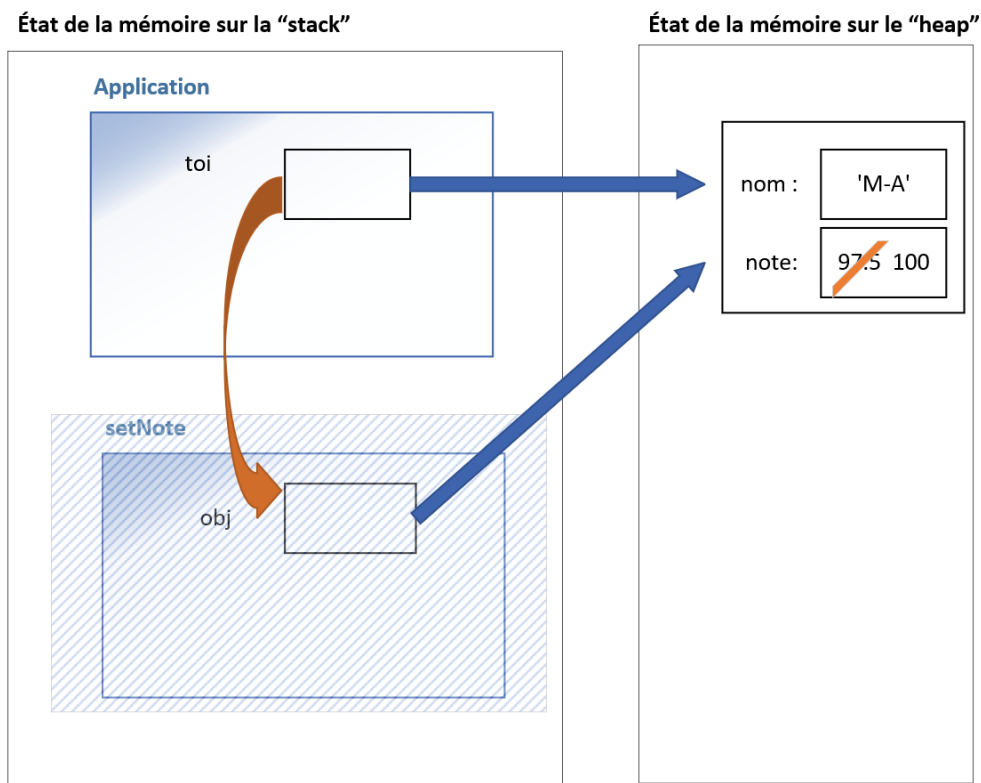
```
public class Etudiant
{
    private String nom;
    private double note;

    Etudiant(String nom, double note)
    {
        this.nom = nom;
        this.note = note;
    }

    public void setNote(double note)
    {
        this.note = note;
    }

    public String toString()
    {
        return this.nom + " - " +
            this.note + "%";
    }
}
```

Lorsque l'objet est passé en paramètre, la méthode reçoit une copie de la référence de l'objet. Cet objet est toujours placé à un endroit unique sur le heap. Il y a donc deux références qui point au même objet sur le heap. L'accès à l'instance, dans la méthode, modifie l'objet sur le "heap" et celui-ci ne sera pas perdue à la fin de la méthode ce qui permettra à l'application de conserver les modifications aux attributs de l'objet.



C'est ce que l'on appelle le passage par référence.